Model updating and data interfaces for industrial robot digital twins

Xiongyule Zhou
School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing 100044, China

Correspondence: jensenzhou47@gmail.com

Article type: Review

Abstract

Industrial robot digital twins can receive live telemetry without updating the models used for prediction and decision making. This distinction matters when assessing whether a published implementation can remain useful after changes in robot condition, tooling or cell configuration. This focused critical review examines how model updates depend on available measurements, interface semantics and validation evidence. It combines targeted literature searches with examination of publicly accessible papers, standards documentation and robot interface specifications, using a search cutoff of 19 October 2026. An update-interface-evidence framework distinguishes state synchronization from geometric and configuration revision, physical parameter identification, and learned residual or surrogate revision. Ten representative implementations are compared with explicit distinctions between full-text evidence and more limited publisher excerpts. The comparison shows why communication performance, calibration accuracy and downstream task performance support different claims. In particular, online inference does not establish online model learning, and a controller-derived Cartesian pose does not independently validate the same controller's kinematic model. Interface selection should therefore begin with the update target and its observability requirements rather than a protocol label or nominal transmission rate. A minimum reporting set links measurement provenance, timestamps, model versions, update triggers and validation conditions. The resulting framework supports more defensible comparison of public cases and identifies practical requirements for reusable datasets and repeatable model-update evaluation. It is an analytical synthesis rather than a new experimental benchmark or an exhaustive systematic review.

Keywords: Industrial robot; Digital twin; Model updating; System identification; Data interface; Validation

1 Introduction

An industrial robot may execute the same program for months while its tool, payload, thermal condition, joint transmission or surrounding fixtures change. A digital representation can remain visually synchronized with that robot even when the model behind its predictions is no longer adequate. This creates a practical problem when visualization evidence accompanies claims about prediction, maintenance or adaptive operation. These capabilities require different measurements and tests. Broad accounts of robot digital twins already describe the enabling technologies and applications in manufacturing [1]. The more specific engineering question is what must change inside a twin, and what evidence establishes that the change improves its intended use.

Existing reviews provide substantial foundations. Böttjer et al. organize unit-level manufacturing twins around reference models, services, content and deployment [2]. Zhang et al. examine iterative updating across equipment twins, including geometric, principle and field models [3]. Sun et al. take a robotic lifecycle perspective [4], while Nele et al. examine machine learning across scales and application domains [5]. These contributions make a further inventory of software platforms or application areas insufficient. The present review connects each update target to its identifying information, interface requirements and validation evidence. This connection is the unit of analysis. Table 1 positions this question against adjacent reviews.

Definitions also require care. The digital model, digital shadow and digital twin classification of Kritzinger et al. emphasizes the automation and direction of data exchange [6]. Jones et al. show that twin descriptions also involve physical and virtual entities, states and metrology [7]. These perspectives are useful but do not, by themselves, distinguish a revised physical parameter from an updated display. This review uses digital twin as the source authors' system designation and records the implemented update separately. It does not retroactively relabel every one-way implementation as invalid. The National Institute of Standards and Technology (NIST) implementation scenarios based on ISO 23247 provide a manufacturing reference context [8]. They provide neither a universal sampling-rate requirement nor a validation certificate.

Three questions guide the synthesis. First, which quantities are updated, and which measurements can support those updates? Second, how do controller interfaces, middleware and model-exchange interfaces constrain what can be inferred and deployed? Third, what does the reported validation establish about synchronization, model adequacy and operational usefulness? These questions connect the mechanics of calibration and identification with the software conditions needed to retain the meaning of a measurement after it leaves a controller.

The contribution is an update-interface-evidence framework and its application to representative public implementations. The framework distinguishes four update targets, separates update timing from communication timing, and links each target to evidence that can support the associated claim. The case synthesis identifies where an implementation demonstrates a component, a connected system or a predictive model. A reporting set then makes the framework usable for future papers and public data releases. These are review-derived analytical proposals; no performance improvement, new experiment or universal ranking of existing systems is claimed.

Table 1 Positioning against adjacent reviews

Review	Primary emphasis	Question pursued here
Qin et al. [1]	Robot architectures, technologies and manufacturing applications	Which update is supported by the connected data and its validation?
Böttjer et al. [2]	Unit-level content, services and deployment	What information must survive the interface for a particular update?
Zhang et al. [3]	Iterative updating of equipment models	How does the update target determine robot-specific sensing and tests?
Sun et al. [4]	Robotic twins across lifecycle phases	How can a reported implementation be separated into update and evidence claims?
Nele et al. [5]	Machine learning across scales and domains	When is online inference distinguishable from model revision?
This review	Update target, data interface and validation evidence	A joint analytical framework with bounded public-case comparison

The final column states the present review's analytical emphasis; it does not assert that adjacent reviews never discuss these issues.

2 Review design and evidence boundaries

2.1 Search and source selection

This is a focused critical review with a documented search trail, rather than a systematic review or meta-analysis. Searches were conducted on 19 October 2026 using a web search service to locate publisher pages, institutional repositories and official technical documentation. Search combinations joined digital twin with industrial robot or robot, and with model updating, calibration, identification, synchronization or Open Platform Communications Unified Architecture (OPC UA). Exact-title searches resolved key records and located accessible versions. Reference examination of the manufacturing robot review [1] and the equipment-updating review [3] provided additional leads. The searches were supplemented by targeted checks of Universal Robots, ABB, Robot Operating System 2 (ROS 2), OPC Foundation, OASIS and Functional Mock-up Interface documentation. No direct Scopus or Web of Science search is claimed. Online Resource 1 records the executed queries and source-selection decisions.

Sources were selected for one of three roles: positioning the review, explaining an update or interface mechanism, or supporting a concrete implementation comparison. For the implementation comparison, a source had to concern an industrial manipulator, a collaborative manipulator used in manufacturing, or an explicitly identified laboratory proxy for such a system. It also had to describe a physical-data connection, a calibration or identification process, or a measurable discrepancy between a model and a physical robot. The ten cases were chosen to cover different update targets and interface arrangements,

not to estimate their prevalence. Purely virtual learning demonstrations, mobile navigation without a relevant manipulator model, and general architectural proposals without a usable mechanism description were not used as implementation evidence. Relevant calibration and identification research was retained even when continuous operation of a connected twin was not demonstrated.

Publisher and Crossref records were used to check titles, authors, publication venues, years and digital object identifiers. Issue years were preferred for bibliographic entries when they differed from early online dates. A DOI establishes the identity of a publication, not the truth or scope of its conclusions. Consequently, substantive claims were checked separately against the available primary text. Unpublished preprints located during searching were not added to the numbered bibliography when a published or accepted version could not be verified. Their exclusion is recorded rather than hidden.

2.2 Extraction and synthesis

Extraction captured the robot and task, update target, measurement source, update mechanism, interface path, timing quantity, validation comparator and availability of supporting artifacts. Full-text PDFs or full publisher HTML were examined for seven of the ten cases. For the other three, only accessible primary publisher abstracts or indexed publisher excerpts were used. These cases are marked P in Table 4; the seven full-text cases are marked F. Absence of a detail in a partial source was coded as not assessable, rather than as evidence that the implementation lacked the capability. Online Resource 2 contains the source-level evidence table with section or page locators and the limits of each extracted claim.

The analysis uses structured comparison rather than a numerical quality score. A well-documented state mirror can provide strong evidence for visualization while providing no evidence for stiffness identification; adding the two into a single score would obscure the intended use. Similarly, calibration error, current-prediction error and interface delay are not commensurate outcomes. Numerical examples are therefore retained with their original endpoint and context, and no pooled accuracy, pooled delay or cross-paper performance ranking is calculated.

The review has several limits. Selection depends on discoverability and public access, and thus favors accessible implementations and documentation. Extraction and interpretation were not independently duplicated by multiple human reviewers. The search was not registered, has no claim of database completeness, and does not support publication-frequency trends. Documentation describes capabilities of a particular interface or software version and does not prove that a paper implemented every capability. Source archives and query records improve traceability but do not remove these limits. The conclusions are intended to clarify evidence requirements and implementation tradeoffs within the examined material.

3 What changes when a robot twin is updated

3.1 State estimates and model changes

A useful distinction begins with a measurement model. In Eq. 1, x is the robot state, theta contains model parameters, c represents configuration and coordinate-frame information,

and epsilon captures measurement error and unmodeled effects. The expression is a conceptual decomposition, not a fitted model of a new dataset.

$$y_k = h(x_k, \theta_k, c_k) + \varepsilon_k \quad (1)$$

Receiving a new encoder vector updates the estimate of x. Replacing a fixture transform changes c. Revising joint stiffness or friction changes theta. A learned model can also change its structure or weights, which requires an identifiable model version in addition to a parameter value. All can be legitimate twin maintenance activities, but they change different objects. Figure 1 shows how these activities share an acquisition path while requiring separate acceptance decisions.

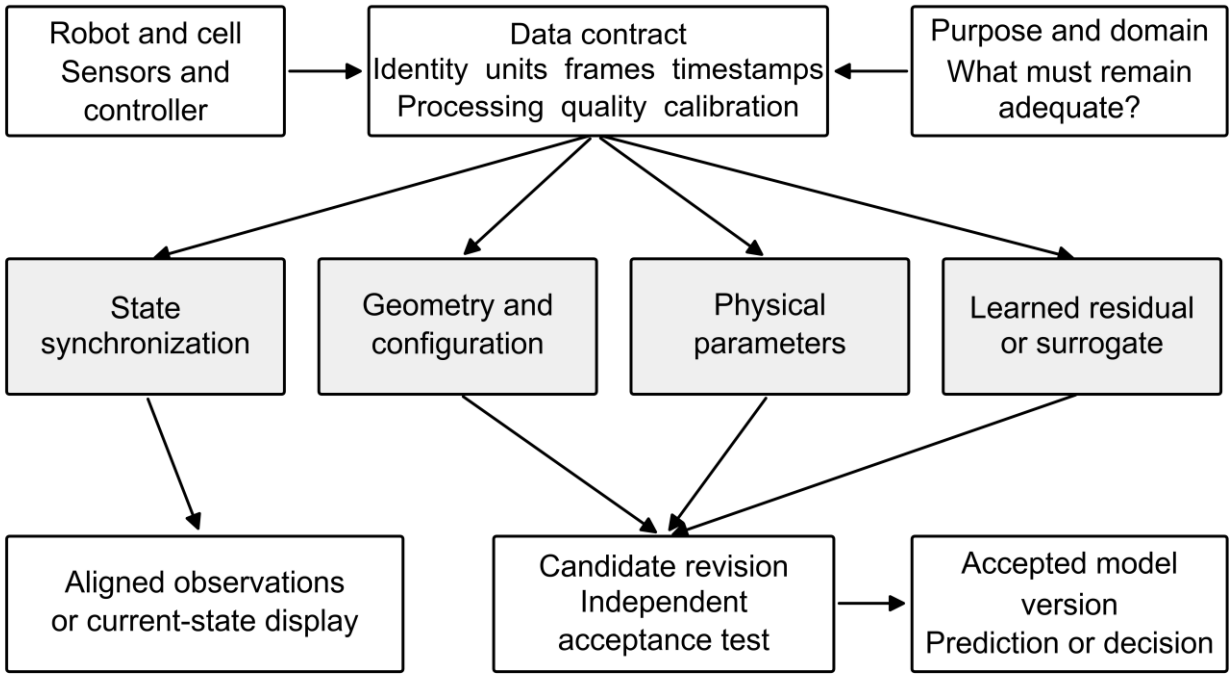


Fig. 1 Shared acquisition path and distinct update targets. State synchronization supplies current observations; model revisions require candidate estimation and acceptance. The schematic is an original analytical synthesis and contains no measured performance data

Robot identification literature supports the importance of this distinction. Lee et al. describe identification as a problem involving model classes, data sufficiency, error criteria and validation [9]. Leboutet et al. compare inertial identification methods and provide the BIRDy toolbox for repeatable evaluation [10]. Neither framework implies that a large volume of routine production telemetry necessarily identifies all physical parameters. Streaming data may repeatedly visit a narrow range of poses and loads. An interface can preserve those samples perfectly while the desired parameter remains weakly constrained by them.

The four targets in Table 2 are analytical categories used in this review. They are not a maturity ladder. State synchronization is sufficient for some monitoring tasks, and a batch-identified physical model may support a prediction task without continuous learning. A single system may implement more than one category.

Table 2 Update targets and their information dependencies

Target	Changed object	Required information	Relevant validation
State synchronization	Current joint, task or environment state	Measurements with ordering, units and usable acquisition times	Aligned state or event comparisons with defined endpoints
Geometry and configuration	Kinematic parameters, frame transforms, tools or cell objects	External pose or constraints; registration; asset and geometry identity	Held-out poses or configurations; reference uncertainty
Physical parameters	Inertia, friction, stiffness, damping or degradation parameters	Excitation, motion and actuation or response data with known processing	Held-out loads, postures or time windows; physical plausibility
Learned residual or surrogate	Weights, residual map or predictive representation	Training and validation domains; target provenance; model version	Separated evaluation data; transfer and drift tests within the claim

Review-derived categories informed by identification and calibration literature [9, 10, 11]. Categories can coexist and do not imply increasing maturity. A new output from fixed weights is inference.

3.2 Geometric and configuration revision

Geometry includes the robot's kinematic parameters and its spatial relationships to tools, cameras, fixtures and workpieces. These quantities should not be conflated. A camera-based revision of a work-object frame can correct a moved station without identifying link lengths or joint-zero offsets. Conversely, robot kinematic calibration does not automatically update an unobserved fixture. The observability work of Borm et al. explains why measurement configuration matters for estimating calibration parameters [11]. The choice of poses can improve information in a finite measurement campaign. Repeated observations of the same configuration may leave ambiguous parameter combinations. Data-driven calibration methods address additional error representations [12], but still need appropriate training and validation observations.

The Universal Robots calibration documentation illustrates a practical representation issue: extracted robot calibration must be handled consistently when generating a robot description for external use [13]. A model that uses nominal geometry while a controller uses robot-specific calibration can disagree even when both sides receive identical joint angles. The review's inference is that model provenance belongs in the interface contract. Recording only the robot model name is insufficient to establish that the same geometry was used in simulation, controller reporting and validation.

Configuration changes can be discrete and event driven. Adding a process module requires its identity, geometry, pose and relevant capabilities to be associated with the same revision. Updating only its mesh leaves collision geometry and task targets

potentially inconsistent. The public Plug & Produce case makes these dependencies concrete, while also showing why validation of the vision component should be distinguished from validation of the entire reconfiguration workflow [14].

3.3 Physical parameter and learned-model revision

Physical updates include inertia, friction, stiffness, damping and degradation-related parameters. The data requirements differ from those of geometric revision. For example, motion and torque-related measurements can constrain inertial or friction parameters, while tool-tip frequency response functions can constrain flexible-joint behavior. Swevers et al. examine excitation design for identification [15], and Wensing et al. show how physical consistency can be imposed through constraints on inertial parameters [16]. These findings motivate retaining excitation conditions and parameter constraints in a twin update record. They do not imply that a single identification procedure is best for every robot or deployment.

Huynh et al. fit stiffness and damping through measured frequency response functions of a milling robot [17]. Hou et al. combine a flexible-joint multibody model with a learned mapping to measured frequency response behavior [18]. The first illustrates revision of interpretable parameters; the second illustrates a hybrid predictive representation. In both settings, the intended output is a physical response across specified conditions. Replaying measured joint angles is not an equivalent validation target.

Learned residuals and surrogates require a further distinction between inference and learning. Using fixed regression weights on a new input updates a prediction, but does not revise the weights. Wang et al. describe a physics-informed current-prediction and anomaly-detection framework [19]. Such a model can be useful without establishing that its deployment includes continual retraining. Likewise, the traceable positioning study explicitly identifies continuous learning from camera measurements as future work [20]. Claims about online adaptation should therefore name the changed parameter or artifact, the incoming evidence, and the rule that accepts the change.

3.4 Timing is an independent dimension

The timescale of a useful update follows the phenomenon and the decision. Joint-state rendering, fixture insertion, thermal drift and long-term degradation need not share a rate. A periodic update, a residual-triggered update and a maintenance-triggered update can each be appropriate under different conditions. The review does not prescribe a universal real-time threshold.

Four timings should be separated. These are the acquisition interval, measurement age at use, estimation duration, and the time needed to accept and activate a model revision. A fast estimator operating on old or misaligned data does not establish timely synchronization. Figure 2 separates these clocks and the different validation endpoints. It also shows why a model may be updated asynchronously while a controller continues operating with a previously accepted version.

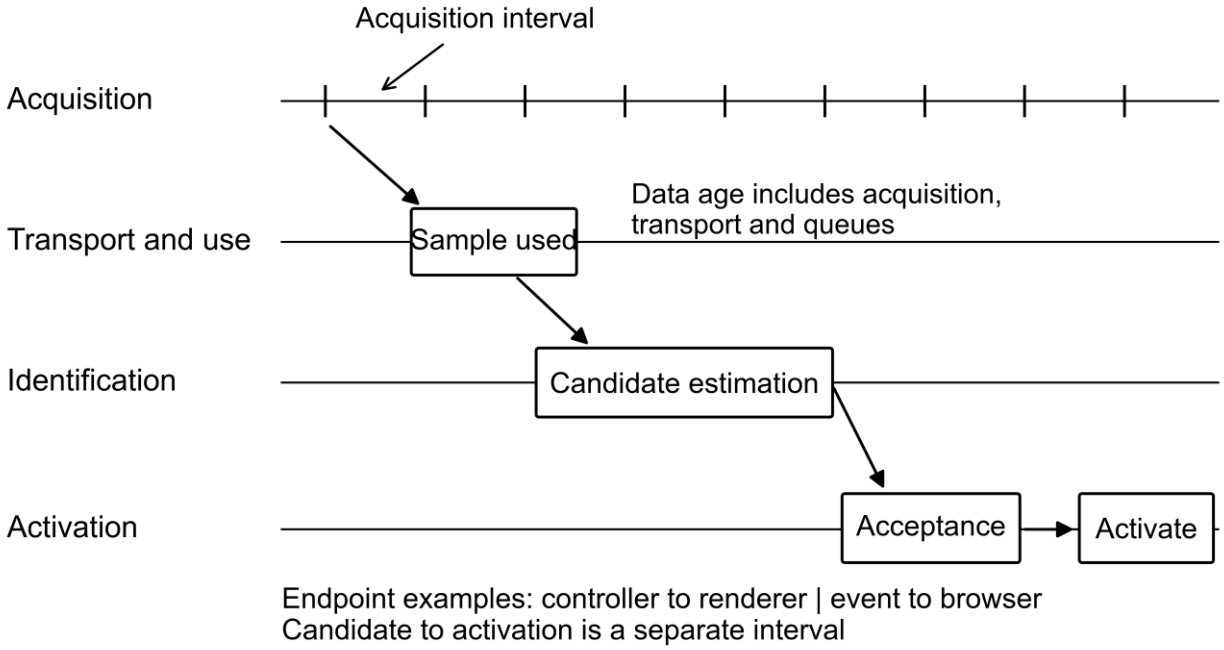


Fig. 2 Acquisition, data use, identification and activation occur on different clocks. Arrows illustrate dependencies rather than measured durations. Delay endpoints and clock relationships must be named before timings can be compared

4 Data interfaces as conditions for valid updating

4.1 Measurement meaning before message transport

An update method can only use information represented in its inputs. For state mirroring, joint position, acquisition time and joint ordering may suffice. For geometric identification, an external pose or constraint and the transformations connecting measurement and robot frames are needed. Dynamic identification additionally depends on motion, load and actuation-related quantities with known interpretation. Current-derived torque, commanded torque and a direct torque-sensor reading should not be silently interchanged. Filtering, gravity compensation and controller estimation can change what each quantity means.

The official Real-Time Data Exchange (RTDE) documentation exposes this problem clearly. Its output fields distinguish joint currents, target moments, current-derived torque and compensated tool forces, with field availability depending on controller software [21]. The documentation describes output at up to 500 Hz for specified robot families, while also allowing skipped output packets under resource constraints. This is an interface capability, not a measurement of end-to-end performance in every installation. An identification dataset should record the selected recipe, software version and observed timestamps, including gaps. Replacing these with the configured rate would turn a deployment assumption into an unsupported fact.

The proposed minimum measurement record is consequently broader than a vector of values. It includes asset identity, signal name, unit, frame or joint index, source timestamp, clock domain, quality flag, relevant controller processing, and model or calibration version.

Not every field must appear in every packet: stable metadata can be referenced through an immutable identifier. What matters is that the receiving process can reconstruct the interpretation that held when the sample was acquired. This is an engineering recommendation derived from the dependencies in Table 2, rather than a claim that a particular communication standard requires this exact record.

4.2 Complementary interface roles

Controller application programming interfaces (APIs), information models, message transports and simulation interfaces solve different parts of the connection problem. Table 3 compares their roles. ABB documents Robot Web Services (RWS) as controller APIs using representational state transfer (REST) and distinguishes versions associated with different RobotWare generations [22]. Such APIs can support state access and supervisory operations, but the allowed endpoints and permissions must be checked for the relevant controller. A diagram labeled REST does not identify which variables or operations were available.

OPC UA for Robotics Part 1 specifies an information model for vertical integration, including configuration and runtime information for motion-device systems [23]. Its value is the structured meaning of the exposed information. It should not be interpreted as a general promise of unrestricted joint-level servo control. A study using a custom OPC UA server also needs to distinguish conformance to the Robotics companion model from merely transporting robot values through OPC UA. Common transport does not establish common signal meaning or an identical operational state machine.

ROS 2 provides typed messages and middleware quality-of-service settings. Reliability, history, deadline and lifespan influence delivery and freshness, but their configuration and compatibility must be reported [24]. The ROS time design also distinguishes system, steady and ROS time, including behavior when simulated time changes [25]. A timestamp is useful for cross-stream alignment only when its clock domain and relationship to other clocks are known. The ros2_control hardware-component lifecycle provides explicit configuration and activation states [26]. These mechanisms support organized integration; their existence does not establish that an application handled reconnection, version changes or stale data correctly.

MQTT provides a standardized publish-subscribe transport with defined delivery quality-of-service levels [27]. It does not supply the robot's kinematic or calibration semantics. In particular, a delivery assurance should not be translated into a claim that a physical action occurred exactly once: command identification, deduplication and acknowledgement of execution are application responsibilities. The Functional Mock-up Interface (FMI) supports exchange and co-simulation of executable models, with version-dependent capabilities such as clocks and scheduled execution [28]. A Functional Mock-up Unit (FMU) can package a model for reuse, but the surrounding application still has to associate it with the correct asset, signals and parameter revision.

Table 3 Interface roles and the information they do not establish by themselves

Interface or specification	Useful role	Additional information needed
UR RTDE [21]	Versioned recipes for controller inputs and outputs	Actual sampling gaps, signal interpretation, calibration and end-to-end timing
ABB RWS [22]	HTTP-based controller access and supervisory operations	Controller generation, API version, permissions and endpoint behavior
OPC UA Robotics Part 1 [23]	Structured asset and runtime information for vertical integration	Implemented model subset, update semantics and any separate control path
ROS 2 and ros2_control [24, 25, 26]	Typed exchange, delivery policies and hardware lifecycle states	Clock relationships, frame conventions, compatible QoS and restart handling
MQTT 5.0 [27]	Publish-subscribe transport and delivery policies	Robot semantics, application freshness and acknowledgement of physical execution
FMI 3.0.2 [28]	Executable model exchange and co-simulation interfaces	Asset binding, parameter provenance, solver configuration and accepted model version

These mechanisms operate at different layers. A standard's capability is not a measured property of a particular robot installation. The table is not a speed ranking. HTTP: Hypertext Transfer Protocol. QoS: quality of service.

4.3 Temporal alignment and apparent model error

Timing affects scientific interpretation as well as responsiveness. For a smooth motion over a small timing offset, the first-order Cartesian discrepancy caused by a joint-state misalignment can be approximated by Eq. 2. J is the kinematic Jacobian evaluated near the operating point, q-dot is joint velocity, and delta-t is the time offset. This local approximation omits higher-order terms and is not an experimental bound.

$$\delta p_{time} \approx J(q)\,\dot{q}\,\Delta t \qquad (2)$$

The implication is specific: a spatial residual observed during motion may partly reflect alignment error. If a fitting process attributes that residual entirely to geometry or friction, it can revise a parameter for the wrong reason. Stationary calibration, explicit delay estimation and separate validation under motion are ways to investigate this ambiguity. Which is appropriate depends on the sensor and task. Neither the expression nor the reviewed cases establish a universally acceptable time offset.

Delay measurements also need named endpoints. Controller-to-renderer delay, camera-to-estimator delay, database ingestion time and event-to-browser display time include different components. Reporting only the most favorable segment is insufficient when the claim concerns a complete observation or decision loop. Conversely, a slow monitoring

dashboard does not imply that the local robot controller is slow. This distinction is essential for interpreting the public examples in Section 5.

4.4 Activating an update is a separate operation

Changing a deployed model has a lifecycle. A candidate revision should be associated with the data window and objective used to produce it, checked against independent validation data where available, and activated with a version identifier. A rollback path and explicit handling of invalid or missing inputs make the transition inspectable. These are proposed design requirements for an auditable update process, not experimentally proven benefits of a new architecture.

The acceptance decision should reflect the update target. A revised fixture transform can be checked against reference geometry and uncertainty. A stiffness model needs response measurements at held-out postures; a learned residual needs trajectories separated from its fitting data. A smaller training residual alone cannot establish improved performance under changed conditions. The twin should also record whether a revision changes prediction only or changes commands sent to a robot. A command-policy update is not automatically a change to the robot's predictive model, and the two need distinct records when both occur.

5 What public implementations establish

5.1 Reading the case comparison

Table 4 compares ten implementations using the same extraction fields. The cases include calibration, response modeling, physical-parameter evolution, state and task synchronization, and a laboratory monitoring proxy. They were deliberately selected to expose different dependencies. F and P indicate the material inspected, not an overall quality grade. The table's limitations concern the evidence available for this review and should not be read as proof that a feature was absent from the original system.

Table 4 Representative implementation comparison

Case and access	Update and data	Implementation path	Evidence boundary
Huynh et al. [17] F	Elastic parameters from tool-tip frequency responses	KUKA KR90 R3100; offline fitting	Physical response fitting; repeated online revision not established
Hou et al. [18] P	Flexible-joint model and learned response mapping	Simulated and measured frequency responses	Mechanism reported; full evaluation and interface not assessed
Aivaliotis et al. [29] P	Tunable physics-based parameters from field data	Selection, revision and propagation methodology	Publisher excerpts inspected; detailed performance not independently assessed

Case and access	Update and data	Implementation path	Evidence boundary
Bennulf et al. [14] F	Process-module pose from robot-mounted vision	REST vision link; RobotStudio SDK/JSON update design	Vision experiment; complete update chain needs separate evidence
Maculotti et al. [20] F	Position error map and correction	Laser tracker, infrared vision, registered frames	Traceability and uncertainty evaluated; continual learning is future work
Zhang et al. [30] F	Offline error model used for compensation	Delta robot, kinematics and drive feedback	Prediction and physical response tested; online use differs from retraining
Liu et al. [31] P	Interface/controller lifecycle coordination	State machines across robot vendors and protocols	Architectural mechanism; detailed performance not assessed
Chang et al. [32] F	State and trajectory synchronization	FANUC interface, TCP/IP and Unity	Laboratory trajectory comparison; physical-parameter learning not demonstrated
Yılmaz et al. [33] F	Synchronization and kinematic/planning comparison	UR10e, ROS 2, Kafka, databases and FMU	Different error endpoints; public dataset landing page verified, data not reprocessed
Rigó et al. [34] F	Local motion mirror and remote task metrics	MiRobot/RoboDK, OPC UA middleware and web path	Laboratory proxy; local and web timings differ

F: full primary text examined. P: primary publisher abstract or indexed publisher excerpts only. These access labels are not quality scores. Absence is not inferred from unavailable text. Section 5 and Online Resource 2 give the locators and limits. SDK: software development kit. JSON: JavaScript Object Notation. TCP/IP: Transmission Control Protocol/Internet Protocol. PLC: programmable logic controller.

Two boundaries organize the synthesis. The first lies between a changed representation and a changed predictive relationship. The second lies between a demonstrated component and a demonstrated operational chain. A camera can accurately estimate a fixture displacement without establishing that every dependent target, collision object and motion program has been updated consistently. A twin can reproduce an observed trajectory without establishing that it predicts an unobserved load or configuration. The following comparisons explain where those boundaries affect interpretation.

5.2 Geometry and spatial correction

The Plug & Produce study by Bennulf et al. describes a chain from module identification and camera-based calibration to simulation revision and path planning [14]. Its experimental section explicitly focuses on the camera-based vision system, while noting that other components had been tested in earlier work. The reported largest single-axis translation error is about 428 micrometers, with average error below 51 micrometers; the application motivation specifies a 300-micrometre requirement. A small average therefore cannot, by itself, establish that the stated tolerance is always met. The defensible conclusion is that the work provides concrete component evidence and a proposed integration path, with the full workflow requiring its own acceptance evidence.

Maculotti et al. provide a complementary emphasis: a laser tracker and an infrared camera system support registration, error modeling and uncertainty propagation for a Yaskawa collaborative robot [20]. Their analysis propagates measurement uncertainty through registration and correction, and reports improved positioning precision under the tested conditions. Real-time error estimation and correction can use an existing model. Continuous learning of the virtual error model is described as future work. The reported gain supports the examined correction procedure and measurement conditions; it should not be generalized to lifelong adaptation or to every payload and trajectory.

Zhang et al. combine a Delta robot twin with an offline error-prediction method and a physical compensation platform [30]. Their results include an algorithm response time and a separate motor-drive response quantity. These describe different stages and should remain separate. The case illustrates how an offline fitted model can support online compensation. It does not require describing each new prediction as a new model update. These examples share dependencies on coordinate registration, measurement uncertainty and the entry point of a revision into the control chain. Display software alone does not specify those dependencies.

5.3 Response models and evolving physical parameters

Huynh et al. offer direct evidence for flexible multibody model updating through tool-tip frequency response measurements and optimization of elastic parameters [17]. The physical robot experiment concerns a KUKA KR90 R3100, and the model and parameterization are chosen for the measured response. This is substantially different from using controller telemetry to animate a nominal rigid robot. The results support the fitting procedure in the tested configurations; they do not establish autonomous repeated identification during production.

The publisher description of Hou et al. reports a flexible-joint multibody model, measured-response fitting and a Gaussian-process mapping between simulated and measured frequency response representations [18]. This connects physical modeling with a learned correction across postures. Access for this review was limited to primary publisher material. The split of postures, complete uncertainty analysis and interface implementation were not independently assessed. The case is retained for the mechanism it describes, with those boundaries visible in the table.

Aivaliotis et al. explicitly address selecting tunable parameters, tuning frequency, field-data collection and propagation of revised values into the virtual model [29]. Their earlier

work links physics-based simulation with remaining-useful-life estimation for an industrial robot [35]. These studies establish that parameter evolution and maintenance are already substantive research topics; novelty cannot be claimed simply by calling for a self-updating twin. The open issue emphasized here is the evidence chain connecting a particular signal, an identifiable parameter change, a deployed version and an independently evaluated downstream prediction. The present review does not reproduce lifetime estimates or treat the papers' reported accuracy summaries as interchangeable benchmark scores.

5.4 Connectivity and synchronization

Liu et al. address establishment and synchronization of multiple robot interfaces using a state-machine-driven approach [31]. Their publisher description emphasizes lifecycle coordination across vendors and protocols. This is a distinct contribution from parameter identification: the connection and controller states must be valid before an update method can receive meaningful data or issue an action. The accessible evidence supports this architectural distinction, while detailed quantitative performance is not assessed here.

Chang et al. implement a robot control and visualization platform with a FANUC interface, Unity and a physical-virtual mapping mechanism [32]. Their trajectory comparison supports behavior within the reported demonstration, but does not identify a repeated physical-parameter learning process. More generally, an implementation can demonstrate useful supervisory control and visualization without supporting a claim that its internal mechanics model evolves with wear. Treating this boundary explicitly makes the system's actual contribution easier to assess.

Yılmaz et al. make a particularly useful distinction among synchronization and tracking error, FMU kinematic prediction error, and path divergence between planning environments [33]. Their joint-state comparison reports cross-correlation-based delay estimates from 0.09 to 15.51 ms, while also listing substantial transient maximum joint errors. The delay estimates should not be reinterpreted as a guaranteed complete-loop deadline. Their data availability statement identifies a versioned Mendeley release; its public landing page was verified for this review, but the recordings were not reprocessed. A public dataset improves inspectability. Independence of the validation reference and the interpretation of safety still require separate assessment.

Rigó et al. provide a laboratory proxy using a MiRobot, RoboDK, a PLC and middleware [34]. Their local virtual-model synchronization is reported below 150 ms, whereas the event-to-web update measurement has a mean of 1.8 s and a maximum of 2.4 s. These measurements can coexist because they concern different paths. The case demonstrates why the phrase real time should be replaced by the measured endpoints, sampling conditions and task tolerance. Its educational manipulator and small laboratory configuration also limit transfer to high-throughput industrial deployment.

5.5 Validation independence and unsupported escalation

A recurring concern is the independence of the comparator. A controller may derive Cartesian position from its own encoders and kinematic calibration. Comparison with another model using the same encoders can reveal representation or implementation differences. It cannot independently establish the robot's absolute tool pose. That stronger

claim needs an external reference or another justified constraint whose errors are characterized. The argument follows from shared inputs and model dependencies; it is not an allegation of error in every controller-based comparison.

Another escalation occurs when an anomaly detector or collision checker is treated as evidence of safety assurance. Detection performance on selected data, successful motion in a demonstration, and compliance of an operational safety function are separate claims. This review does not assess certification. It asks authors to retain the scope of the validation actually performed and to report the operating envelope relevant to a downstream decision. Credibility work by Shao et al. likewise links verification, validation and uncertainty to the intended use of a manufacturing twin [36].

Table 5 maps common claims to the evidence needed to interpret them. The table is a review-derived reporting aid, not a standard and not a retrospective pass-fail instrument. It makes explicit why a convincing interface demonstration can still leave parameter accuracy or deployment robustness unresolved.

Table 5 Claims and the evidence needed to interpret them

Claim	Evidence to report	Insufficient substitute
The twin is synchronized	Named endpoints, clock relationship, sampling behavior and data-age distribution	Configured rate or animation refresh rate alone
The model is updated	Changed parameter, transform or artifact; trigger; data window; accepted revision	A new prediction generated by fixed weights
Position accuracy improves	Reference independence, frame registration, metric definition and operating conditions	Agreement between two poses derived from the same encoders and calibration
A model generalizes	Held-out poses, loads, sessions or configurations matched to the claim	A smaller fitting residual on the update data
The complete workflow works	An observed chain from physical change to accepted model and task execution	Separate component demonstrations without a chain test
The implementation is reproducible	Accessible versioned artifacts, dependencies and evaluation scripts	A repository link without sufficient data or a stated execution environment

Proposed reporting aid synthesized from the case comparison and credibility principles [36]. It is not a certification checklist.

6 Implications for implementation and reusable evaluation

6.1 Choosing the smallest adequate update

The case comparison supports choosing the update target before the software stack. A moved fixture calls first for a revised spatial relationship and associated collision and

target geometry. Increasing the neural-network size does not resolve an unknown frame transform. Conversely, a torque discrepancy that changes with velocity or load may require a dynamic or residual model, while repeated geometric calibration alone may not address it. This is a diagnostic ordering based on information dependencies, rather than a claim that one method class dominates another.

For limited sensing, the appropriate scope may be narrower than a complete twin. Encoder-only data can support state alignment and internal consistency checks. An external pose reference adds information relevant to geometric correction. Motion and actuation data under suitable excitation add information for dynamic identification. Force or response measurements can address compliance, while temperature and operating history can help separate changing conditions from an apparent fault. No extra signal guarantees identifiability: the link to the parameter and the operating conditions must still be established. The modeling perspective of Rasheed et al. is consistent with treating model fidelity and data integration as joint engineering problems [37].

Figure 3 translates these dependencies into a selection and acceptance workflow. It begins with a specified decision and measurable failure of adequacy, and ends with activation of a revision only within a documented domain. A failed acceptance test can justify collecting different data, reducing the parameter set or retaining the previous model. It need not trigger an increasingly complex learner. The workflow is proposed for practical use and remains to be evaluated prospectively.

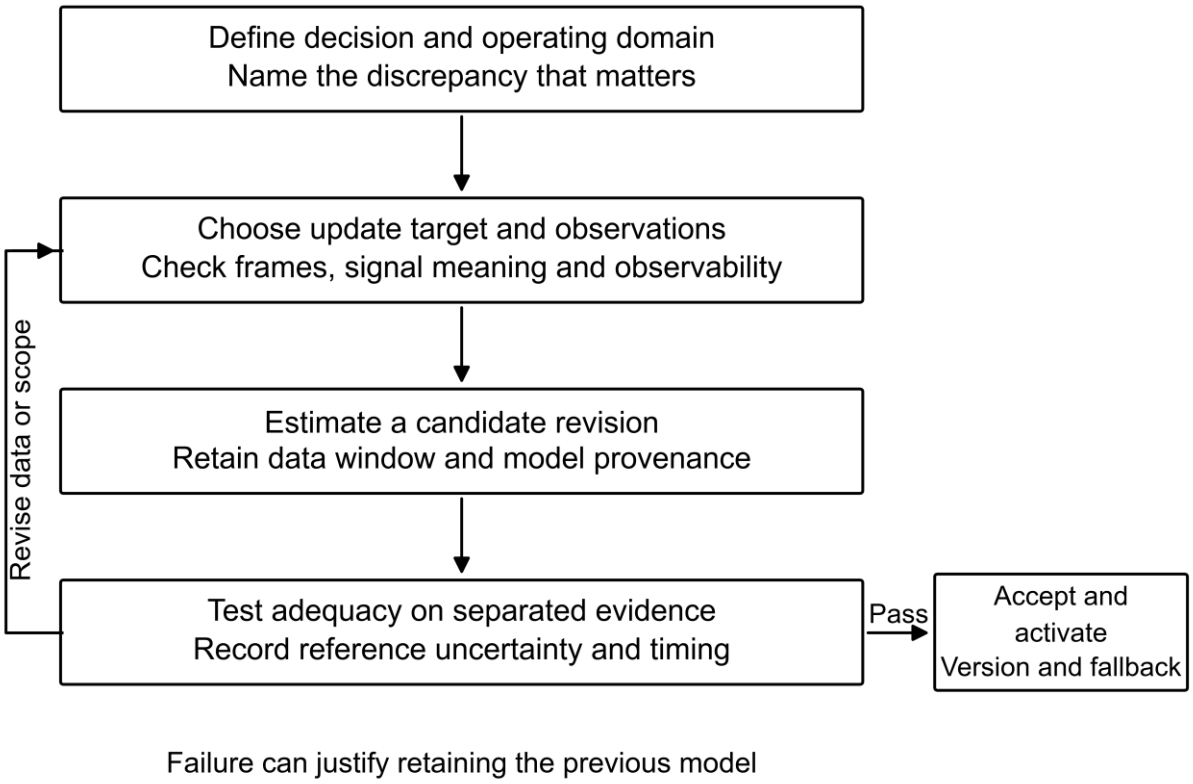


Fig. 3 Proposed workflow linking the intended decision to an accepted model revision. An unsuccessful test returns the process to the choice of data or update scope. This is a review-derived proposal requiring prospective evaluation

6.2 A minimum reporting set

Table 6 specifies the reporting set derived from this review. Each item connects a claim to an inspectable artifact. A paper that claims model updating should identify the revised object, the trigger, the data used to revise it, and the artifact activated afterward. A paper that claims synchronization should give the timing endpoints, clock relationship, acquisition settings and behavior under gaps. A paper that claims predictive improvement should distinguish fitting data from validation data and explain whether the reference shares the model under test.

Reporting should also preserve failed or rejected updates when relevant to the claimed method. A process that accepts only favorable updates can appear robust if rejected candidates and their causes are omitted. This observation motivates keeping acceptance logs; it does not imply an observed rejection frequency in the examined literature. Similarly, the availability of a repository should be reported separately from evidence that another party reproduced the results. Source code, parameter files and recordings are complementary artifacts, and none alone establishes independent replication.

Table 6 Minimum reporting set for an auditable model update

Item	Record or artifact	Interpretive purpose
Decision and domain	Intended output; task tolerance; poses, loads and environment	Defines what adequacy means
Update object	State, transform, physical parameter or learned model; units and constraints	Distinguishes revision from inference
Measurement provenance	Sensor/controller fields; calibration; processing; reference independence	Preserves the meaning of the evidence
Temporal provenance	Acquisition time, clock domain, alignment and gaps	Separates temporal and model discrepancy
Interface implementation	Controller/API versions, schema, frame and joint conventions, QoS	Allows the data path to be reconstructed
Update procedure	Trigger, data window, objective, prior or regularization, execution duration	Makes the inference step inspectable
Acceptance and activation	Held-out test, acceptance rule, accepted version and fallback	Separates candidate revision from deployment
Availability	Data and model versions, scripts, dependencies and access restrictions	Distinguishes public description from reusable artifacts

Review-derived proposal. Items should be proportionate to the stated claim; no fixed update rate or universal accuracy threshold is prescribed.

6.3 Public data that would resolve current ambiguities

Reusable evaluation requires data organized around a change in the physical system and a corresponding model revision. A useful release would pair pre-change and post-change recordings with the changed tool, payload, fixture or condition identified. It would include characterized measurement channels, calibration transforms, timestamps and clock information. Nominal and revised models, fitting and held-out trajectories, and evaluation scripts would complete the release. These requirements are more informative for update evaluation than a large collection of unlabeled steady-state logs.

Different tests answer different questions. A held-out posture probes spatial transfer; a held-out payload probes load transfer; a later session probes temporal stability; and a reconnection sequence probes interface continuity. Testing all of them is unnecessary for a narrowly stated claim, but none can stand in for all the others. The BIRDy identification benchmark provides one example of reusable evaluation infrastructure at the method level [10]. The versioned dataset linked by Yılmaz et al. provides a separate example of making synchronized robot recordings discoverable [33]. Neither is treated here as a complete benchmark of lifelong twin updating.

The next useful comparative study would therefore hold the measurement and validation conditions fixed while varying the update mechanism. It could compare a fixed calibrated model, periodic identification and event-triggered revision, with the same held-out trajectories and explicit accounting for computation and data age. This is a research proposal, not an experiment performed in this review. The expected benefit is an interpretable comparison: any improvement could be attributed more clearly to the update rather than to a different sensor, reference frame or timing convention.

7 Discussion

The central result is a dependency argument. An update claim requires its target, identifying information, delivery conditions and validation endpoint to refer to the same physical and computational situation. Increasing connection bandwidth does not resolve a missing external reference. Adding a detailed mechanics model does not resolve an unknown time offset. A low average residual does not, on its own, establish compliance with a worst-case tolerance. These observations explain why comparing published implementations solely through platform names or headline accuracy is unreliable.

The framework extends existing review emphases rather than replacing them. Broad robot architectures [1] explain where enabling technologies fit; equipment-updating taxonomies [3] explain classes of model revision. The present synthesis connects the revised quantity to its measurements, interface metadata and supporting test. This makes the evidence requirements explicit for a particular update claim. Its practical value is in making an implementation choice or a paper's evidence boundary visible. The proposed categories have not been validated as a standardized assessment instrument, and agreement between independent coders remains an appropriate future test.

Three cautions limit generalization. First, the ten cases are illustrative rather than a representative statistical sample, so the review cannot estimate the proportion of industrial twins that update physical parameters. Second, the partial-access cases support narrower conclusions than the full-text cases. More complete access could change their

classification or reveal additional implementation detail. Third, a public description of a successful experiment leaves deployment questions unresolved when loads, controller versions, sensing arrangements or operating environments change. These limits are why the review avoids an aggregate maturity score and a universal recommendation for a protocol or learning method.

For embodied intelligence, the same distinction is consequential. Updating a policy in simulation, revising the environment geometry and identifying the robot's physical dynamics are related but separate operations. A learned controller may benefit from a revised twin, yet task success alone does not identify which model component became more accurate. Keeping model and policy versions, physical change records and independent evaluation conditions together would allow future work to connect twin maintenance with reliable learning more convincingly. This is a bounded implication of the information dependencies, not evidence that the reviewed systems solve general embodied adaptation.

8 Conclusions

This review distinguishes live state synchronization from revision of geometry and configuration, physical parameters, and learned residual or surrogate models. The comparison of ten public implementations shows how these targets lead to different data needs, interface responsibilities and validation claims. Evidence for connectivity, component calibration and downstream operation should be reported at the level actually demonstrated.

The main practical recommendation is to define the update target and its identifying measurements before choosing the interface. Timestamp meaning, coordinate frames, signal processing and model versions are part of the evidence chain. Online prediction should be distinguished from online learning, and an internally computed pose should not be presented as an independent reference for the same kinematic model. The proposed reporting set and workflow provide a concrete basis for more comparable papers and reusable data releases. Their prospective evaluation, together with broader literature coverage and independent extraction, is needed before treating them as a validated assessment method.

Statements and declarations

Funding

[Author to complete and verify the funding statement before submission.]

Competing interests

[Author to complete and verify the declaration of financial and non-financial competing interests before submission.]

Author contributions

[Author to enter the actual author list and contributions before submission.]

Ethics approval and consent to participate

Not applicable. This review reports no new experiments involving human participants or animals and no analysis of identifiable participant data.

Consent for publication

Not applicable.

Data availability

No new robot measurements were collected and no source dataset was reprocessed. The review's search record and evidence extraction are supplied as Online Resource 1 and Online Resource 2. The underlying publications and technical documentation are identified in the reference list. Local source copies are retained for verification and are not included in the public supplementary files.

Supplementary information

Online Resource 1 Search record and source-selection decisions, including the executed queries, access limitations and the scope of the focused review.

Online Resource 2 Literature evidence table with source roles, update mechanisms, validation locators and claim boundaries. Entries are extracted review records, not new experimental observations.

References

1. Qin Q, Liu Z, Zhong R, Wang XV, Wang L, Wiktorsson M, et al. Robot digital twin systems in manufacturing: Technologies, applications, trends and challenges. Robotics and Computer-Integrated Manufacturing. 2026;97:103103. https://doi.org/10.1016/j.rcim.2025.103103

2. Böttjer T, Tola D, Kakavandi F, Wewer CR, Ramanujan D, Gomes C, et al. A review of unit level digital twin applications in the manufacturing industry. CIRP Journal of Manufacturing Science and Technology. 2023;45:162-189. https://doi.org/10.1016/j.cirpj.2023.06.011

3. Zhang B, Ding G, Zheng Q, Zhang K, Qin S. Iterative updating of digital twin for equipment: Progress, challenges, and trends. Advanced Engineering Informatics. 2024;62:102773. https://doi.org/10.1016/j.aei.2024.102773

4. Sun Y, Fu Z, Li P, Xu Y, Xiong Z, Ji J. Robotic Digital Twin: A Lifecycle Perspective from Design to Application and Maintenance - A Review. CHAIN. 2026;3(1):37-52. https://doi.org/10.23919/chain.2026.000004

5. Nele L, Mattera G, Yap EW, Vozza M, Vespoli S. Towards the application of machine learning in digital twin technology: a multi-scale review. Discover Applied Sciences. 2024;6(10):502. https://doi.org/10.1007/s42452-024-06206-4

6. Kritzinger W, Karner M, Traar G, Henjes J, Sihn W. Digital Twin in manufacturing: A categorical literature review and classification. IFAC-PapersOnLine. 2018;51(11):1016-1022. https://doi.org/10.1016/j.ifacol.2018.08.474

7. Jones D, Snider C, Nassehi A, Yon J, Hicks B. Characterising the Digital Twin: A systematic literature review. CIRP Journal of Manufacturing Science and Technology. 2020;29:36-52. https://doi.org/10.1016/j.cirpj.2020.02.002

8. Shao G. Use Case Scenarios for Digital Twin Implementation Based on ISO 23247. Gaithersburg: National Institute of Standards and Technology; 2021. NIST AMS 400-2. https://doi.org/10.6028/nist.ams.400-2

9. Lee T, Kwon J, Wensing PM, Park FC. Robot Model Identification and Learning: A Modern Perspective. Annual Review of Control, Robotics, and Autonomous Systems. 2024;7(1):311-334. https://doi.org/10.1146/annurev-control-061523-102310

10. Leboutet Q, Roux J, Janot A, Guadarrama-Olvera JR, Cheng G. Inertial Parameter Identification in Robotics: A Survey. Applied Sciences. 2021;11(9):4303. https://doi.org/10.3390/app11094303

11. Borm JH, Meng CH. Determination of Optimal Measurement Configurations for Robot Calibration Based on Observability Measure. The International Journal of Robotics Research. 1991;10(1):51-63. https://doi.org/10.1177/027836499101000106

12. Chen T, Yang W, Li S, Luo X. Data-Driven Calibration of Industrial Robots: A Comprehensive Survey. IEEE/CAA Journal of Automatica Sinica. 2025;12(8):1544-1567. https://doi.org/10.1109/jas.2025.125237

13. Universal Robots. Calibration correction algorithm [Internet]. Universal Robots ROS 2 Driver Documentation. [cited 2026 Oct 19]. Available from: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_robot_driver/ur_calibration/doc/algorithm.html

14. Bennulf M, Ramasamy S, Zhang X, Danielsson F, Swathanandan J. Automatic Calibration and Update of a Digital Twin for Plug & Produce. Sensors. 2025;25(22):6885. https://doi.org/10.3390/s25226885

15. Swevers J, Ganseman C, Tukel DB, de Schutter J, Van Brussel H. Optimal robot excitation and identification. IEEE Transactions on Robotics and Automation. 1997;13(5):730-740. https://doi.org/10.1109/70.631234

16. Wensing PM, Kim S, Slotine JJE. Linear Matrix Inequalities for Physically Consistent Inertial Parameter Identification: A Statistical Perspective on the Mass Distribution. IEEE Robotics and Automation Letters. 2018;3(1):60-67. https://doi.org/10.1109/lra.2017.2729659

17. Huynh HN, Assadi H, Dambly V, Rivière-Lorphèvre E, Verlinden O. Direct method for updating flexible multibody systems applied to a milling robot. Robotics and Computer-Integrated Manufacturing. 2021;68:102049. https://doi.org/10.1016/j.rcim.2020.102049

18. Hou M, Cao H, Shi J, Guo Y. An industry-oriented digital twin model for predicting posture-dependent FRFs of industrial robots. Mechanical Systems and Signal Processing. 2024;212:111251. https://doi.org/10.1016/j.ymssp.2024.111251

19. Wang S, Tao J, Jiang Q, Chen W, Qin C, Liu C. A digital twin framework for anomaly detection in industrial robot system based on multiple physics-informed hybrid convolutional autoencoder. Journal of Manufacturing Systems. 2024;77:798-809. https://doi.org/10.1016/j.jmsy.2024.10.016

20. Maculotti G, Khusnuddinov F, Kholkhujaev J, Genta G, Galetto M. Traceable digital twin for accurate positioning of industrial robot arms in human-robot collaborative systems. Flexible Services and Manufacturing Journal. 2025. Advance online publication. https://doi.org/10.1007/s10696-025-09632-7

21. Universal Robots. Real-Time Data Exchange (RTDE) Guide [Internet]. PolyScope Tutorials. [cited 2026 Oct 19]. Available from: https://docs.universal-robots.com/tutorials/communication-protocol-tutorials/rtde-guide.html

22. ABB. Robot Web Services and IoT Gateway [Internet]. Robotics downloads. [cited 2026 Oct 19]. Available from: https://www.abb.com/global/en/areas/robotics/downloads

23. OPC Foundation. OPC UA for Robotics Part 1 Vertical Integration [Internet]. OPC 40010-1. Version 1.02. [cited 2026 Oct 19]. Available from: https://reference.opcfoundation.org/specs/OPC-40010-1/full

24. Open Robotics. Quality of Service settings [Internet]. ROS 2 Jazzy documentation. Version Jazzy. [cited 2026 Oct 19]. Available from: https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html

25. Open Robotics. Clock and Time [Internet]. ROS 2 design. [cited 2026 Oct 19]. Available from: https://design.ros2.org/articles/clock_and_time.html

26. ros2_control contributors. Lifecycle of a Hardware Component [Internet]. ros2_control documentation. Version Jazzy. [cited 2026 Oct 19]. Available from: https://control.ros.org/jazzy/doc/ros2_control/hardware_interface/doc/lifecycle_of_a_hardware_component.html

27. OASIS. MQTT Version 5.0 [Internet]. OASIS Standard. Version 5.0. [cited 2026 Oct 19]. Available from: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html

28. Modelica Association. Functional Mock-up Interface Specification [Internet]. FMI Standard. Version 3.0.2. [cited 2026 Oct 19]. Available from: https://fmi-standard.org/docs/3.0.2/

29. Aivaliotis P, Arkouli Z, Georgoulias K, Makris S. Methodology for enabling dynamic digital twins and virtual model evolution in industrial robotics - a predictive maintenance application. International Journal of Computer Integrated Manufacturing. 2023;36(7):947-965. https://doi.org/10.1080/0951192x.2022.2162591

30. Zhang Y, Gao P, Wang Z, He Q. Research on status monitoring and positioning compensation system for digital twin of parallel robots. Scientific Reports. 2025;15(1):7432. https://doi.org/10.1038/s41598-025-91892-2

31. Liu Z, Liu S, Wang T, Wang L, Wang XV. Establishment and Synchronisation of Digital Twins for Multi-robot Systems in Manufacturing. Procedia CIRP. 2025;134:419-424. https://doi.org/10.1016/j.procir.2025.02.152

32. Chang W, Sun W, Chen P, Xu H. Construction and Validation of a Digital Twin-Driven Virtual-Reality Fusion Control Platform for Industrial Robots. Sensors. 2025;25(13):4153. https://doi.org/10.3390/s25134153

33. Yılmaz M, Yılmaz CS, Kahraman S, Yayan U. Real-Time Big Data Pipelines for Industrial Robot Digital Twins: An OMPL Benchmarking Framework. Machines. 2026;14(6):702. https://doi.org/10.3390/machines14060702

34. Rigó L, Fabianová J, Kovalčík J. Digital Twin and Virtual Monitoring of a 6-DOF Robotic Manipulator. Logistics. 2026;10(9):205. https://doi.org/10.3390/logistics10090205

35. Aivaliotis P, Georgoulias K, Chryssolouris G. The use of Digital Twin for predictive maintenance in manufacturing. International Journal of Computer Integrated Manufacturing. 2019;32(11):1067-1080. https://doi.org/10.1080/0951192x.2019.1686173

36. Shao G, Hightower J, Schindel W. Credibility consideration for digital twins in manufacturing. Manufacturing Letters. 2023;35:24-28. https://doi.org/10.1016/j.mfglet.2022.11.009

37. Rasheed A, San O, Kvamsdal T. Digital Twin: Values, Challenges and Enablers From a Modeling Perspective. IEEE Access. 2020;8:21980-22012. https://doi.org/10.1109/access.2020.2970143